

INGEST BASELINE · RELIABILITY

1. 从“能采上来”到“可重复采集”

采到一次不是能力，能稳定复现才是能力。

这节课把 Week02 的 contract / manifest / gate 推进到 Week03 的运行时现实：state、run evidence 与恢复路径。

开场定调

Week03 地图

追踪锚点

四类事故

工程入口

行业信号

Week03 学习链

01

Reliability

为什么 ingest 可靠性决定下游

02

Batch

幂等写入、重跑与完整性校验

03

Incremental / CDC
cursor、watermark、
checkpoint

04

Asset Flow

用 Dagster 组织 ingest、分区与回
放

05

Recovery

Replay / Backfill / Runbook

WHAT THIS LESSON FIXES

这节课先解决什么问题

WHY IT MATTERS

为什么它重要

- Week02 已经定义哪些输入允许进入。
- 但允许进入，不等于它能稳定进入 lake。
- 重复、缺口、错位、静默丢失会先污染下游。
- 没有 state 与 run evidence，失败后只能靠人肉回忆。

WHAT YOU CAN DO

学完至少能做到

- 解释 ingest baseline 的 5 个条件。
- 区分 manifest、batch、run 与 state。
- 用 4 类事故定位输入链路问题。
- 找到 repo 里的 seed_loader、manifest、contract tests。
- 写出一份最小 smoke report。

WEEK03 MAP

先把 Week03 的地图看清



本周一句话

把 Week02 的 contract / manifest / gate, 收成一条可运行、可重跑、可补数、可回放、可解释的采集与入湖基线。

下游关系

Week04 lakehouse
Week06 orchestration/recovery
Week08 RAG evidence serving
Week11+ eval/governance

PART 01

先立生产级判断

系统不是采不到才危险，而是看似能采，却无法解释和复现。

Lesson 01

输入运行不是“脚本跑完”，而是 run 边界、state 和恢复路径都成立。

批次

状态

证据

KEY THESIS

生产里最危险的不是采不到，而是采到了却复现不了

“今天能采，明天漂，后天想重跑却找不到证据。”

真正的 ingest 风险，不是一次失败，而是系统一直把不可解释的输入继续向下游传播。

RUN

执行可重复

同一批次失败后能安全再执行，而不是越跑越脏。

STATE

状态可持久化

run_id、checkpoint、cursor、watermark 不能只在日志里。

EVIDENCE

结果可解释

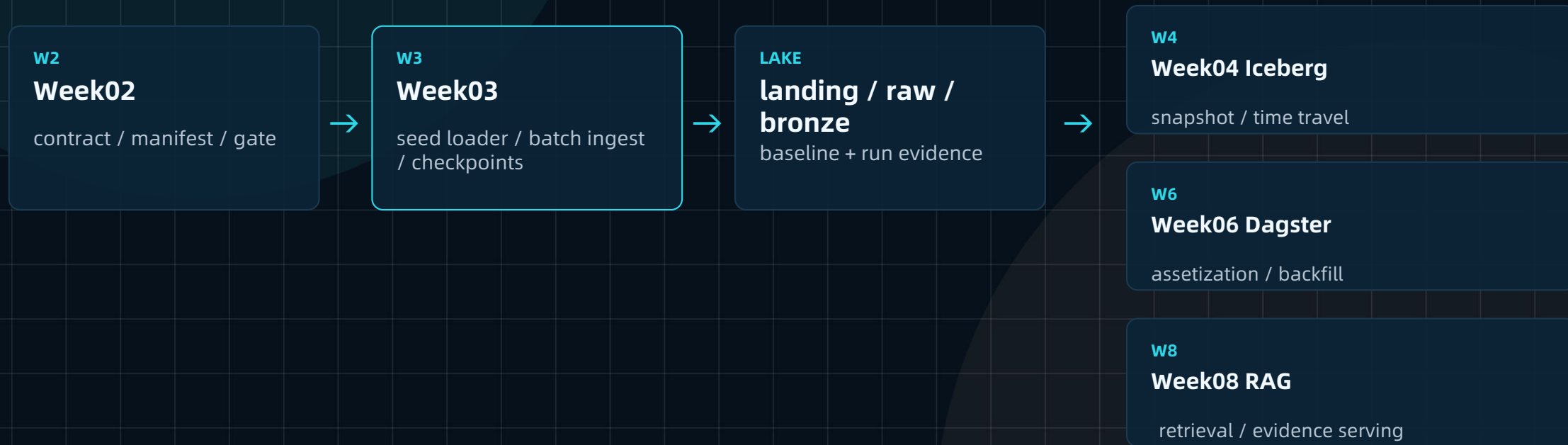
知道本次接了谁、写了多少、跳过什么、为什么。

RECOVERY

恢复有路径

retry、replay、backfill 不是临时救火脚本。

讲义核心图：Week02 → Week03 → lake baseline



NOT ANOTHER ETL

为什么 Week03 不是 “再讲一次 ETL”

常见误解	Week03 真正关心	生产后果
只是能读文件	manifest / source / batch window 是否明确	不知道这次到底接了谁
只是写进去	写入语义是否幂等、可追踪、可恢复	重跑后重复、缺口、污染
测试通过	run evidence 能否解释执行事实	失败后无人能复盘
追求实时	先建立可靠性基线	越快地传播错误

COMPARISON

“能采上来” vs “可重复采集”

维度	能采上来	可重复采集
执行方式	脚本跑完一次	同一批次可安全再执行
数据边界	隐含在路径或参数里	manifest / source / batch window 显式声明
失败恢复	重跑全量或人工补	retry / replay / backfill 有边界
对账能力	看日志猜测	total / valid / invalid / inserted / skipped 可核对
下游影响	污染后才发现	gate / report 提前留下证据
复盘能力	靠口头经验	run evidence 可回放、可解释

INGEST BASELINE

什么叫 Week03 的 ingest baseline

它不是完整采集平台，而是一组最低运行时承诺。

01

输入边界明确

manifest / source /
batch window

02

执行可以重复

rerun 不制造额外副作用

03

状态可以持久化

run_id / checkpoint /
cursor / watermark

04

结果可以解释

run evidence / report

05

恢复有路径

retry / replay / backfill

这 5 条少一条，后面的 lakehouse、RAG、eval 都会踩在不稳定输入上。

TRACE ANCHORS

先建立 5 个必须被带起来的追踪锚点

锚点不是日志装饰，而是恢复和审计的入口。

manifest_id

这次运行引用哪份输入声明。

batch_id

这批数据的业务/时间边界。

run_id

这一次执行的唯一身份。

source_fingerprint

原始来源有没有变化。

trace_id

跨模块追踪与排障链路。

先问有没有锚点，再问有没有工具。

STATE OBJECTS

再拆 3 个状态对象：checkpoint / cursor / watermark

CHECKPOINT

checkpoint

状态被落在哪里：文件、表、asset metadata，还是只有日志。

CURSOR

cursor

下一次从哪里继续读：updated_at、offset、LSN、sequence。

WATERMARK

watermark

系统已经确认处理到哪里：用于迟到、乱序和补数判断。

没有 state, replay 和 backfill 只能靠猜。

REALITY CHECK

如果今晚 ingest 失败，第二天你最需要知道什么

这一页把抽象能力压成排障问题。

哪个 manifest 在跑

不是“哪个脚本”，而是本次输入声明。

跑到哪个 source

失败发生在读取、校验、写入还是对账。

哪些已经写入

需要知道已写入、已跳过、已隔离。

哪些未写入

缺口能否被 bounded replay/backfill 覆盖。

下游看到什么

旧数据、新数据、还是半成品状态。

恢复动作是什么

retry、replay、backfill 不能混用。

这 6 个问题回答不出来，就不算有 ingest baseline。

PART 02

把风险讲成可复盘的事故

从重复、缺口、错位和无追踪，看输入链路怎么悄悄坏掉。

四类事故

Duplicate / Gap / Mismatch / No Traceability, 是 Week03 的排障母题。

重复

缺口

漂移

FOUR INCIDENT CLASSES

输入链路里最常见的 4 类事故

DUP

Duplicate

重放、重试、cursor 粒度不足导致重复。

GAP

Gap

窗口错位、失败未补、source 漏读。

DRIFT

Mismatch / Drift

schema 合法但业务语义变了。

TRACE

No Traceability

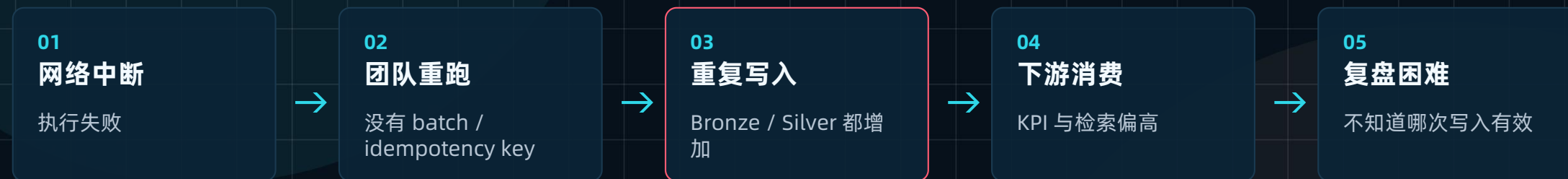
无法回到 manifest / run / source。

事故复盘不靠“感觉”，靠锚点、状态和报告。

INCIDENT A · DUPLICATE

重复不是小问题：它会污染统计、检索和工具动作

重复常常来自“恢复动作不带幂等”。



屏幕上看起来像什么

- 报表数值变高一点
- RAG chunk 里出现重复内容
- 工单事件看似只是多了一条

实际已经坏在哪

- retry 变成重复副作用
- dedupe key / idempotency key 缺位
- run evidence 无法解释哪次写入有效

INCIDENT B · GAP

缺口比失败更隐蔽：系统可能继续跑，但少了一段历史

Gap 常常来自 window / cursor / checkpoint 不一致。



屏幕上看起来像什么

- 系统没报错
- 数据量只是少一点
- 下游报告仍能生成

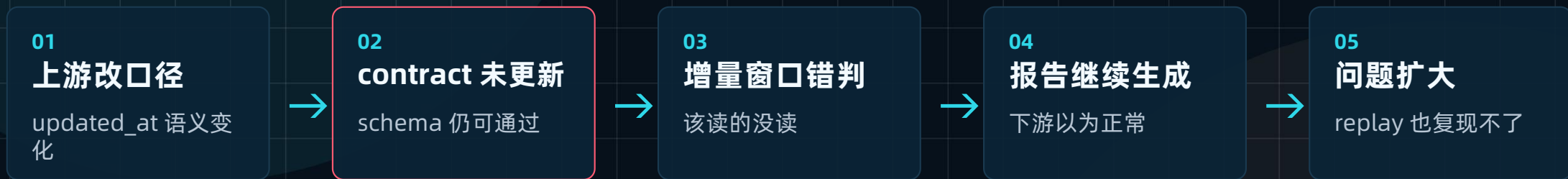
实际已经坏在哪

- state 先于事实前进
- watermark 与写入结果不一致
- backfill 范围难以界定

INCIDENT C · MISMATCH / DRIFT

错位/漂移：字段合法，但运行时意义已经变了

这是 Week02 语义漂移在 Week03 的运行时版本。



屏幕上看起来像什么

- 字段还在
- schema 仍过
- 只是“偶尔不准”

实际已经坏在哪

- load semantics 漂移
- cursor 失去业务含义
- 需要 contract + manifest + state 一起查

INCIDENT D · NO TRACEABILITY

无追踪：采集成功了，但没人能解释这次到底发生了什么

这是生产排障时最伤团队信任的状态。



屏幕上看起来像什么

- 日志里有 done
- 表里有数据
- 终端看起来成功

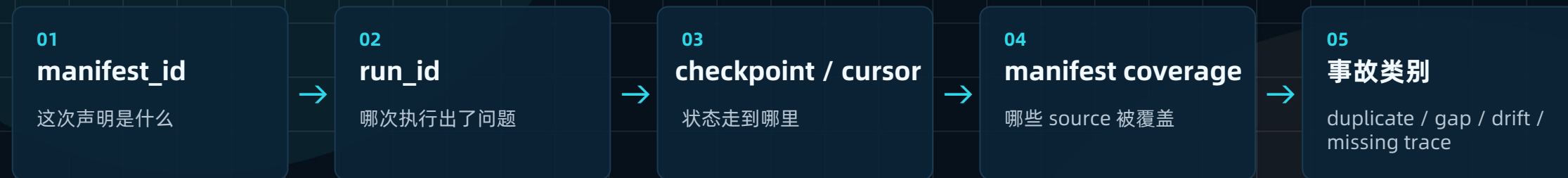
实际已经坏在哪

- 没有 run evidence
- 没有 source_fingerprint
- 没法证明“这批数据”是什么

DIAGNOSE FIRST

Week03 排障顺序：先找锚点，再谈修复

不要一上来重跑。先判断边界、状态、覆盖率。



判断口令：先定位，后恢复；先边界，后命令。

REPO REALITY

把 Week02 的 contract / manifest 接到 Week03 的 repo 现实里

CONTRACT

contracts/data/*.json

四类输入 contract

MANIFEST

data/seed_manifests/*.json

source manifest 与 schema

LOADER

pipelines/ingestion/seed_loader.py

dry-run 入口

TESTS

tests/contract/test_json_schemas.py

契约测试

REPORT

reports/week03/*.md

smoke / recovery report

DOCS

docs/blueprints/week03/*.md

baseline / state 说明

Week03 第一周就要把“运行证据”写进交付物。

动手实践：跑通一次最小 ingest baseline smoke flow

```
docker compose --env-file infra/env/.env.local \
-f infra/docker-compose.yml up -d --build

docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
python -m pipelines.ingestion.seed_loader \
--manifest-dir data/seed_manifests

docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
pytest tests/contract/ -v
```

OBSERVE

观察 1

读到了哪些 manifest / source

OBSERVE

观察 2

contract tests 是否是门禁

WRITE

写下来

ingestion_baseline_v1.md +
ingest_smoke_report.md

SMOKE REPORT

一份 ingest_smoke_report 至少要写什么

报告是 run evidence 的最小形态。

Run Identity

run_id / manifest_id / git_sha /
command

Source Coverage

哪些 source 被读取、跳过、隔离

Gate Result

accept / quarantine / reject / warn +
reason

State Snapshot

checkpoint / cursor / watermark 当
前值

Observed Counts

total / valid / invalid / inserted /
skipped

Next Action

continue / retry / replay / backfill

没有报告, dry-run 只是“看起来跑过”。

INDUSTRY SIGNALS

行业正在把 run evidence 变成对象

OpenLineage

Facets

Run / Job / Dataset 上挂 metadata; 适合承载 how it ran、what was used、what outcome occurred。

Airbyte

Incremental reality

增量同步依赖 cursor, at-least-once 现实意味着重复不是异常。

Dagster

Asset materialization

运行不只是日志, 资产产出和 metadata 会成为后续观察入口。

Course Repo

Docker-first baseline

统一 devbox 命令, 避免“我本机能跑”的偶然成功。

共同趋势：运行意图和运行证据正在对象化。

BRIDGE FROM WEEK02

Week02 工件在 Week03 里分别变成什么

把“规则文件”压成“运行时对象”。

Contract

从字段规则变成 admission gate

Manifest

从装车单变成 batch boundary

Gate action

从 pass/fail 变成分流路径

Run evidence

从观察记录变成恢复依据

Week03 不是推翻 Week02，而是消费 Week02。

RECAP & NEXT

这节课真正完成了什么

先把 ingest reliability 的心智立住，再进入 Batch 主链路。

本课最重要的判断

- ingest 成功一次，不等于 ingest 可靠。
- manifest 决定这次接什么。
- state 决定接到哪里。
- report 决定到底发生了什么。

继续向前

- duplicate / gap / drift / no trace 是四类事故。
- replay / backfill 需要前置，不是救火补丁。
- Week03 不是实时化冲动，而是可靠性基线。
- 下一讲开始把 batch 主链路做稳。

下一讲

Lesson 02 会把 manifest-driven batch ingest、幂等写入、完整性校验和 reconcile 做成最小主链路。

BATCH INGEST · IDEMPOTENCY

2. 批量采集主链路

Week03 不是先追求快，而是先追求稳。

批量链路是最适合讲幂等、重跑、完整性校验和对账的入口。

Batch First

幂等写入

重跑设计

完整性校验

Reconcile

工程入口

Week03 学习链

01

Reliability
为什么 ingest 可靠性决定下游

02

Batch
幂等写入、重跑与完整性校验

03

Incremental / CDC
cursor、watermark、checkpoint

04

Asset Flow
用 Dagster 组织 ingest、分区与回放

05

Recovery
Replay / Backfill / Runbook

WHAT THIS LESSON FIXES

这节课先解决什么问题

manifest 写了，不代表 batch ingest 天然可重跑。

WHY IT MATTERS

为什么它重要

- 路径能读，不代表批次边界清楚。
- 写入成功，不代表重复执行没有副作用。
- dry-run 通过，不代表真实写入就可靠。
- 只有完整性校验和 reconcile 才能证明结果对得上。

WHAT YOU CAN DO

本课你应该拿走

- 解释幂等写入、重跑、完整性校验、reconcile 的边界。
- 读懂 ticket_ingest.py 的最小能力。
- 说清 Bronze / Silver 的写入语义。
- 能设计一份 batch ingest 说明。

本课目标：让 batch 主链路从“脚本能跑”变成“可安全重跑”。

WHY BATCH FIRST

为什么先从 batch 讲起

边界最清晰

source + window +
batch_id 可显式声明

最适合幂等

同批次重复执行能观察
副作用

最适合校验

total / valid / invalid /
inserted 对得上

最适合恢复

天然连接 rerun / replay
/ backfill

最适合教学

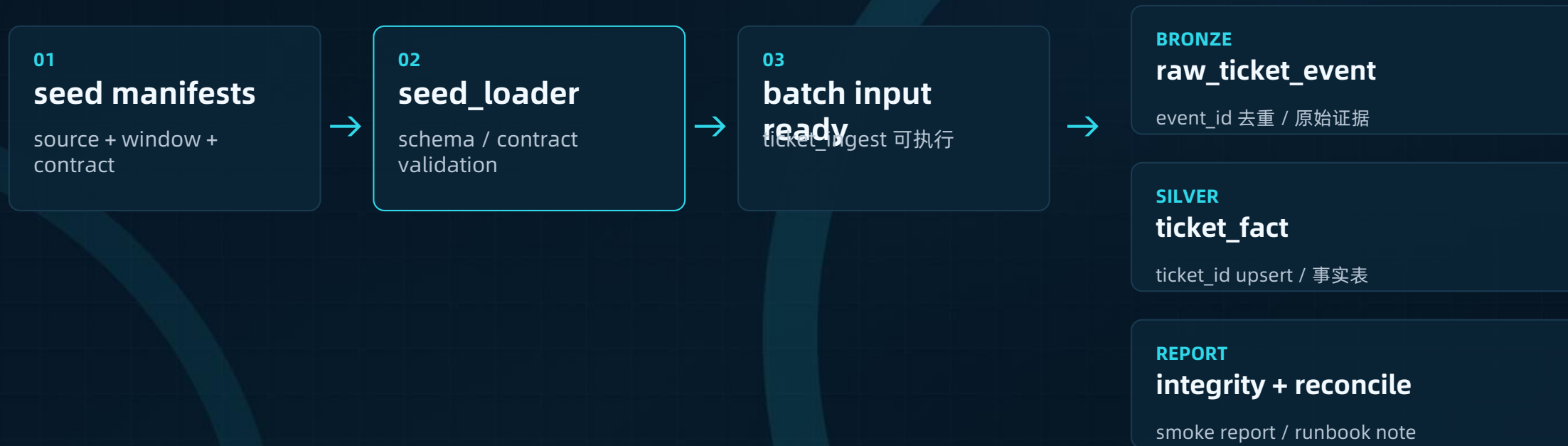
小白能先建立完整链路
心智

先做 batch，是为了让后续 incremental / CDC 有可靠参照物。

LECTURE DIAGRAM · REDRAW

讲义核心图：batch ingest 主链路

seed manifests 到 Bronze/Silver, 再到 reconcile/report.



这张图的重点不是画法，而是每一步都能留下 run evidence。

CONCEPT SPLIT

四个概念先拆开：不要混用

它们共同成立，batch 链路才算可靠。

概念	它解决什么	没有它会怎样
幂等写入	重复到达/重复执行时不制造额外副作用	重跑后越写越脏
重跑 rerun	同一条 job 失败后能安全再执行	失败恢复变成赌博
完整性校验	写完后验证数量、错误、跳过是否合理	写完了但不知道写对没
reconcile	manifest 声明、写入结果、异常记录对上	少/多/错都无法解释

这四个不是同义词，是 batch baseline 的四道门。

BATCH BOUNDARY

先把 batch 边界写清楚

manifest_id

本次输入声明是谁

batch_id

这批数据的业务边界

window_start / end

时间窗口是否闭合

source_fingerprint

原始输入是否改变

边界写不清，后面幂等、对账、补数都会失真。

IDEMPOTENT WRITE

幂等写入保护的是重复执行的副作用

“能重复执行”必须先问：重复写入会不会产生副作用？

生产里 retry 很常见，真正危险的是 retry 把错误放大成重复数据。

BRONZE

event_id 去重

raw_ticket_event 保留事件证据，但不能无限重复。

SILVER

ticket_id upsert

ticket_fact 维护当前事实，不是追加垃圾。

KEY

idempotency key

写入动作重复时判断是不是同一副作用。

AUDIT

保留原因

skipped / inserted / updated 要能解释。

COMPARISON

Bronze / Silver 写入语义不同，幂等策略也不同

不要把所有层都当成 append。

层级	写入目的	幂等判断
Bronze raw_ticket_event	保存事件/来源证据	event_id 或 source_id+event_time 去重
Silver ticket_fact	形成可消费事实表	ticket_id upsert, 保留更新时间
Report / State	解释本次运行结果	run_id + batch_id 唯一化
Quarantine	隔离坏记录	reason_code + source pointer 可复盘

幂等不是“一律 upsert”，而是每一层有自己的写入承诺。

重跑设计：同一批次失败后能否安全再执行一次

有幂等不等于一定可重跑，但想可重跑通常先要有幂等。

同一批次

batch_id / manifest_id 不变，避免把重跑当新数据。

同一输入

source_fingerprint 不变，才能判断复现。

同一状态

checkpoint / report 能解释从哪里恢复。

同一策略

contract / gate action 不要悄悄变化。

可观察输出

inserted / skipped / errors 要能比较。

可记录结论

runbook 说明为什么 rerun，而不是 replay/backfill。

重跑不是“再敲一次命令”，而是同一批次的可控再执行。

COMPLETENESS CHECK

完整性校验：写完了不等于写对了

至少解释六个数。

total

输入总量

valid

合同内合法记录

invalid

坏记录

inserted

新增写入

skipped

幂等跳过

errors

执行错误

这六个数不对，别急着说 batch 成功。

RECONCILE

Reconcile: manifest 声明、写入结果、异常记录能否对上

reconcile 不是对账软件，而是工程解释能力。

对账对象	要问的问题	失败信号
Manifest coverage	manifest 里的 source 都被处理了吗	source missing / skipped without reason
Source count	输入记录量与读取量一致吗	total 不一致
Valid / Invalid	合法、非法、隔离是否闭合	$\text{valid} + \text{invalid} \neq \text{total}$
Bronze / Silver	写入层之间是否能解释	Bronze 有, Silver 缺
Unexplained gap	有没有未解释缺口	report 无 reason_code

Reconcile 的目标不是“数值漂亮”，而是“缺口可解释”。

RERUN VS REPLAY

先把 rerun 和 replay 分开

本课重点是 rerun；Lesson05 会把 replay/backfill 系统讲透。

动作	对象	典型场景	风险
rerun	同一条 job	执行中断、依赖服务短暂失败	没有幂等就重复写
replay	同一批输入	验证幂等、重放 source 批次	输入版本混淆
backfill	历史范围/分区	补历史空洞或重算旧分区	影响范围过大

一句话：rerun 是执行级再跑，replay 是输入级重放。

PART 02

把 batch 链路落到 repo

从 seed_loader 到 ticket_ingest, 再到 tests 和 report。

工程入口

不要先发明新架构。先读当前 repo 里已有的三类对象。

loader

ingest

tests

REPO REALITY

当前 repo 三个核心对象

LOADER

`pipelines/ingestion/seed_loader.py`

读取 manifest, 做最小 dry-run

DATA

`data/seed_manifests/*.json`

source / contract / window 声明

INGEST

`pipelines/ingestion/ticket_ingest.py`

ticket JSONL → Bronze/Silver

CONTRACT

`contracts/data/*.json`

字段、证据、边界规则

TESTS

`tests/contract/test_json_schemas.py`

contract / manifest schema 验证

REPORTS

`reports/week03/*.md`

smoke / integrity / recovery

TICKET_INGEST.PY

ticket_ingest.py 的最小能力卡

它不是终极 pipeline, 但已经足够支撑 batch 可靠性教学。

JSONL 输入

读取结构化工单事件

contract 校验

对齐 ticket_contract

业务校验

最小 status / time / tenant 规则

Bronze 写入

raw_ticket_event

Silver 写入

ticket_fact

--batch-id

显式批次身份

--dry-run / --limit

先观察, 不盲写

summary 统计

total / valid / inserted / errors

DRY-RUN VS REAL WRITE

dry-run 很重要，但不是终局

dry-run 与真实写入面对的风险不同。

阶段	它能验证什么	它验证不了什么
dry-run	manifest / schema / contract / 大致路径	DB constraint / transaction / upsert conflict
real write	实际写入、约束、错误处理	是否有完整解释，仍要看 report
integrity check	数量、错误、跳过是否闭合	业务语义是否完全正确
reconcile	manifest 与结果是否对上	上游未来是否继续稳定

QUIETLY FAIL

batch 链路最容易 quietly fail 的地方

字段合法不代表语义正确，写入成功不代表可消费。

status 合法但语义漂

新增状态没有同步下游统计/路由。

updated_at 口径不清

发生时间、更新时间、入湖时间混用。

ticket_id 非稳定主键

upsert 对错对象。

Bronze 写成，Silver 不全

下游以为有事实表，实际缺行。

dry-run 通过

真实写入被 constraint 或 transaction 拦。

report 太粗

只说 done，不说 skipped/errors。

WRITE PITFALLS

真实写入时会遇到的工程坑

这一页体现实战经验：不是所有失败都在 schema 层。

DB constraint

唯一键、非空、外键约束触发

upsert conflict

冲突策略不明确，覆盖历史证据

transaction failure

半批写入导致状态不一致

clock drift

窗口边界和更新时间错位

retry storm

错误重试放大重复写

partial commit

Bronze/Silver 不一致

quarantine missing

坏记录直接丢失

weak report

无法解释结果

DESIGN TEMPLATE

batch ingest 设计说明模板：先回答 5 个问题

source 是谁

source_id / owner /
contract_ref

batch 边界是什么

batch_id / window /
source fingerprint

idempotency key 是什么

event_id / ticket_id /
run+sink key

integrity check 看什么

total / valid / invalid /
inserted / skipped

失败后怎么办

rerun / replay / backfill
的选择

ENGINEERING HANDOFF

工程 handoff: 跑 batch baseline 的最短路径

命令用于演示，讲解重点是观察输出。

```
docker compose --env-file infra/env/.env.local \
  -f infra/docker-compose.yml up -d --build

docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  pytest tests/contract/ -v

docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.ingestion.seed_loader \
  --manifest-dir data/seed_manifests

# ticket_ingest dry-run
python -m pipelines.ingestion.ticket_ingest \
  --dry-run --limit 20 --batch-id week03-smoke
```

CHECK**先看 tests**

contract 是否仍是门禁

CHECK**再看 loader**

manifest 是否可解释

CHECK**最后看 ingest**

summary 是否闭合

OUTPUT OBSERVATION

看到 summary 时，你到底在看什么

Total

输入量

Valid

通过校验

Invalid

被拦截

Inserted

实际写入

Errors

执行异常

Total $\neq$ Valid $\neq$ Inserted，这三个差异就是对账入口。

RUN REPORT FIELDS

batch run report 至少应该带这些字段

identity

run_id / batch_id / manifest_id

input

source_id / fingerprint / window

write stats

inserted / updated / skipped / errors

quality stats

valid / invalid / reason_code

state

checkpoint before / after

reconcile

coverage / gap / unexpected

operator note

为什么 rerun/replay

next action

continue / fix / backfill

INDUSTRY SIGNALS

行业信号：重复是现实，幂等是底座

不要把 duplicate 当偶发异常。

Airbyte

at-least-once

Incremental Append 明确可能产生同主键多版本记录；重复是运行语义的一部分。

Dagster

asset metadata

materialization metadata 让一次产出成为资产级证据。

OpenLineage

run facets

run evidence 可以附着到运行上下文，帮助事后定位。

Course stance

Batch as baseline

批量链路不是落后，而是可靠性的教学基线。

幂等、dedupe、reconcile 是生产采集的基本功。

COMMON MISUNDERSTANDINGS

本课最容易误解的 5 件事

误解

正确理解

Batch 很简单，不值得讲

Batch 是可靠性基线，最适合训练幂等与对账。

幂等 = 去重

幂等保护写入副作用；去重判断输入是否重复。

dry-run 通过就能上线

dry-run 只证明准入路径初步成立。

重跑就是 replay

rerun 是执行级，replay 是输入级。

写入成功就结束

还要完整性校验与 reconcile。

MINI EXERCISE

课堂小练习：给 ticket ingest 写 5 行设计说明

让学生把概念压成自己的工程判断。

Source

ticket_events_synthetic

Batch

2026-04-13 daily
window

Idempotency

event_id for bronze /
ticket_id for silver

Integrity

total-valid-invalid-
inserted-skipped

Failure action

rerun first, replay if input
needs re-consume

RECAP & NEXT

Batch 主链路完成了什么

从“脚本能跑”推进到“批次可解释、可重跑”。

本课最重要的判断

- Batch 是 Week03 最稳入口。
- 幂等写入、重跑、完整性校验、reconcile 不是同义词。
- dry-run 重要，但不是终局。
- Bronze / Silver 写入语义不同。

继续向前

- summary 统计必须能解释缺口。
- run report 是后续 replay/backfill 的前置证据。
- quietly fail 比显式失败更危险。
- 下节课把 batch 推到 incremental / CDC 语义。

下一讲

Lesson 03 会把 cursor、watermark、checkpoint、CDC 和 exactly-once 的边界讲清楚。

INCREMENTAL · CDC · STATE

3. 增量与 CDC

看起来像增量，不等于已经有稳定增量链路。

这节课把 cursor、watermark、checkpoint、乱序、去重和 exactly-once 的边界一次讲清。

cursor

watermark

checkpoint

CDC

dedupe

not exactly-once

Week03 学习链

01 Reliability
为什么 ingest 可靠性决定下游

02 Batch
幂等写入、重跑与完整性校验

03 Incremental / CDC
cursor、watermark、
checkpoint

04 Asset Flow
用 Dagster 组织 ingest、分区与回放

05 Recovery
Replay / Backfill / Runbook

WHAT THIS LESSON FIXES

这节课先解决什么问题

增量真正难在状态与恢复，不在“少读一点数据”。

WHY IT MATTERS

你会遇到的问题

- 哪个字段配当增量游标？
- updated_at 为什么危险？
- WAL / logical decoding / CDC 解决什么、没解决什么？
- 迟到、重复、乱序怎么处理？
- 为什么不要轻易承诺 exactly-once？

WHAT YOU CAN

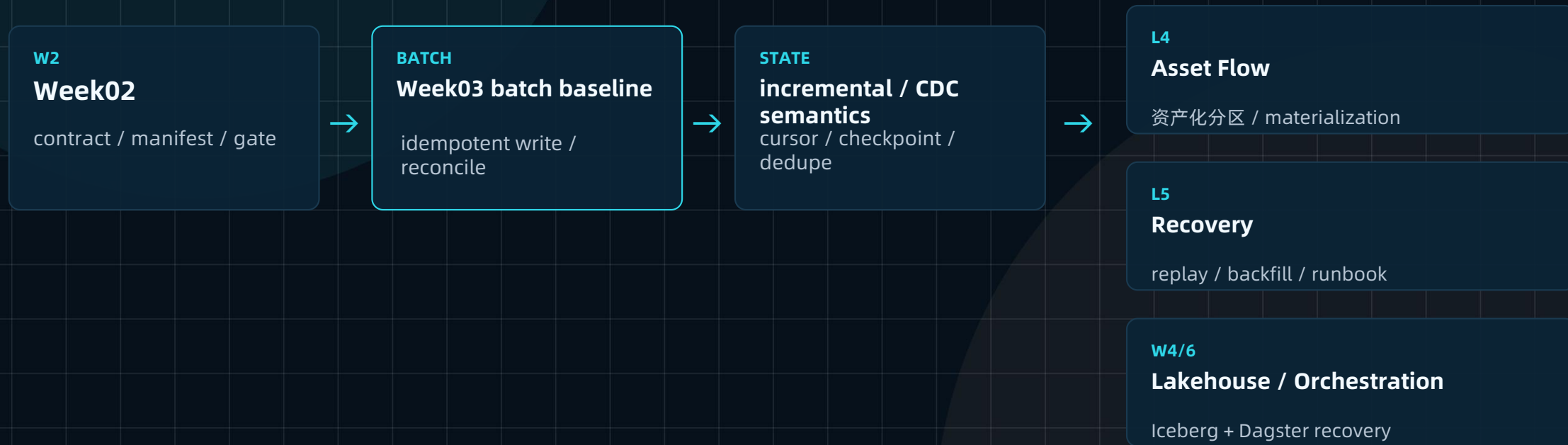
本课目标

- 拆开 cursor / watermark / checkpoint。
- 区分 dedupe key 与 idempotency key。
- 理解 at-least-once 的现实。
- 能写出 incremental_ingest_strategy_v1.md。
- 对 CDC 的边界保持工程诚实。

LECTURE DIAGRAM · REDRAW

讲义核心图：从 batch baseline 到 incremental / CDC

增量不是跳过 batch，而是在 batch 可靠性上继续叠状态。



WHY HARDER

为什么增量比全量更难

全量简单在“边界大但清楚”；增量难在“边界小但持续变化”。

风险	全量 / Batch	Incremental / CDC
重复	通常来自重跑	cursor 粒度、slot 重发、retry 都可能重复
迟到	一次性批次较容易观察	watermark 之前/之后要有策略
乱序	窗口内排序影响小	事件顺序与业务语义可能错位
恢复	重跑整个批次	必须知道 checkpoint / LSN / offset
解释	一次 run report	持续状态 + 分区 report

OFFICIAL SIGNAL · AIRBYTE

Airbyte 的增量同步信号：cursor 与 at-least-once 是现实

用官方文档帮助学生建立边界感。

cursor

cursor 决定是否复制

cursor 是用来判断记录是否应该被增量复制的值。

append

Append 会保留多版本

更新记录会被追加，不会原地修改。

at-least-once

重复可以发生

Airbyte 提供 at-least-once 复制保证，重复不是异常。

limitation

cursor 粒度不足会出事

日级 cursor、多次修改、updated_at 未更新都可能导致重复或漏变更。

工程结论：增量首先是边界声明问题，不是速度问题。

5 CONCEPTS

5 个概念必须拆开

每个概念回答不同问题。

cursor

下一次从哪里继续读

watermark

当前承认处理到哪里

checkpoint

这个边界落在哪里

dedupe key

两条输入是否同一事件

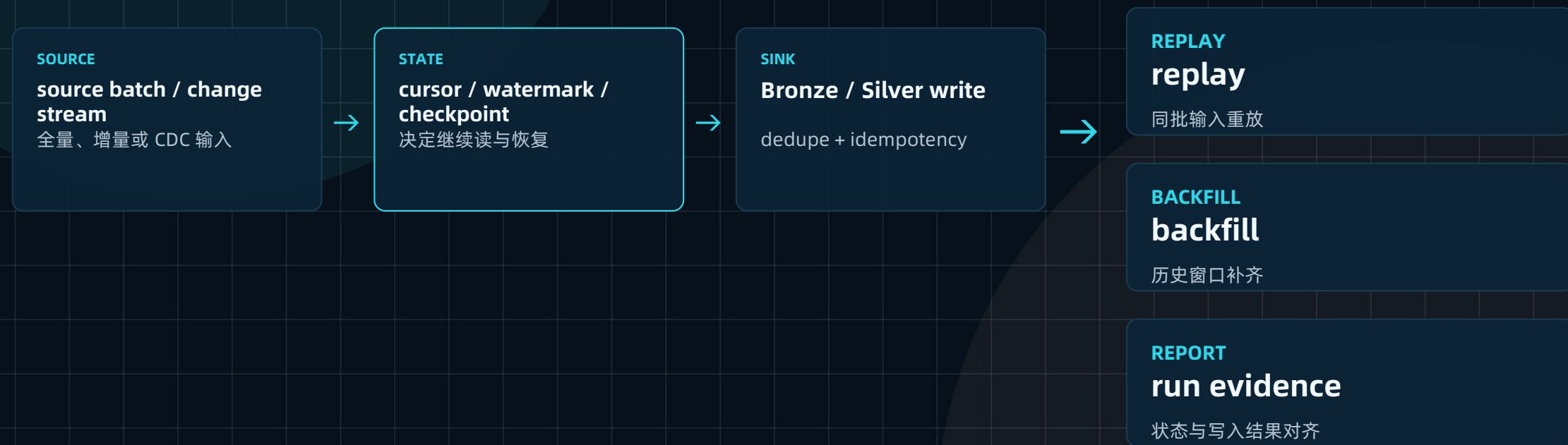
idempotency key

重复写入是否有副作用

混用这些词，是增量系统最常见的设计味道。

讲义状态结构图：读取层、状态层、写入层

同一条 change stream 要同时走状态和写入两条逻辑。



CURSOR FIELD CHOICE

cursor 字段怎么选：每种都有风险

字段能当 cursor，前提是它真的表达变化边界。

字段	适合场景	主要风险
updated_at	常规数据库表增量	被回写、没更新、粒度太粗
event_time	事件发生时间	迟到和乱序会打穿窗口
sequence_id	单调递增业务序列	需要来源保证单调
LSN	数据库 WAL/CDC	偏日志位置，不等于业务语义
offset	消息/日志流位置	只代表读取位置，不代表数据事实

不要问“哪个字段好”，先问来源是否保证它的语义。

WATERMARK VS CHECKPOINT

watermark 和 checkpoint 不要混

一个是承认边界，一个是落盘状态。

对象	回答什么	没有它会怎样
watermark	系统已经确认处理到哪里	迟到/乱序无法判断
checkpoint	这个边界被持久化在哪里	crash 后不知道从哪恢复
cursor	下一次从哪里继续读	增量变成全量或漏读
report	这次处理发生了什么	state 与事实无法对账

没有 checkpoint, crash recovery、replay、backfill 都靠记忆。

KEY SPLIT

dedupe key vs idempotency key: 输入层和写入层不要混

两个 key 都重要，但保护的边界不同。

Key	判断对象	例子
dedupe key	两条输入是不是同一业务事件	event_id / ticket_id+updated_at / source_id+LSN
idempotency key	同一次写入动作重复执行是否有副作用	batch_id+primary_key / run_id+sink_key
primary key	目标表里一个事实对象如何定位	ticket_id / doc_id+version
trace key	跨系统如何串联排障	trace_id / run_id / manifest_id

写入层幂等不能替代输入层去重。

UPDATED_AT PITFALLS

updated_at 为什么危险

它很常见，但不是天然可靠。

被回写

批处理或同步任务修改时间戳。

没更新

数据变了，但 cursor 字段没变。

粒度太粗

一天内多次变化无法区分。

语义漂移

从业务更新时间变成 ETL 时间。

时区不统一

窗口边界错位。

NULL / default

默认值导致新旧难分。

源端延迟

变更晚于同步窗口出现。

并发写入

多个更新顺序不可控。

updated_at 可以用，但必须配合窗口、容忍区、dedupe 和 report。

LATE ARRIVAL

迟到、重复、乱序怎么做第一轮决策

不要把所有异常都 reject。

场景	默认动作	要记录什么
迟到但在 watermark 容忍窗口内	accept + mark late	event_time / observed_at / reason
超出 watermark 容忍窗口	quarantine or backfill review	source_id / window / gap
dedupe key 重复	skip or merge	original event pointer
idempotency key 重复	skip write side effect	existing sink key
schema 合法但语义漂移	quarantine + review	contract / owner / downstream impact

动作背后要有 reason_code，否则后面无法回放。

PART 02

CDC 解决什么，没解决什么

WAL/slot/LSN 能提高变化捕获能力，但不会自动消灭重复和恢复边界。

工程诚实

CDC 是 snapshot + change stream 的组合，不是 exactly-once 魔法。

WAL

slot

LSN

POSTGRES LOGICAL REPLICATION

PostgreSQL logical replication: 先 snapshot, 再持续 changes

CDC 不完全取代 snapshot, 而是把初始状态与后续变化接起来。

publication / subscription

发布者声明变化, 订阅者消费变化。

initial snapshot

先复制已有数据, 建立起点。

continuous changes

之后持续发送
INSERT/UPDATE/DELETE。

transaction order

同一 subscription 内按发布顺序应用。

logical decoding 与 slot: 能重放, 也可能重发

官方文档明确说: crash 后最近 changes 可能再次发送。

logical decoding

从 WAL 中抽取持久化变化。

replication slot

代表可重放的 change stream。

crash-safe but...

位置只在 checkpoint 持久化。

client responsibility

客户端要避免重复处理副作用。

这就是为什么 CDC 客户端仍要 dedupe + idempotent write。

DEBEZIUM SIGNAL

Debezium exactly-once: 不要轻易承诺

官方文档提供了非常适合课堂的工程边界。

default

默认 at-least-once

不漏 change, 但 record 可能多次 delivery。

no internal dedupe

没有内部 dedup layer

Debezium 自身不实现内部去重层。

Kafka Connect EOS

可利用 Kafka Connect EOS

需要 distributed mode、Kafka Connect 版本和配置前置条件。

known issues

仍有边界

官方文档提示 exactly-once 的正确性仍需谨慎看待。

不要轻易承诺 exactly-once

工程上更诚实的目标，是可复现、可定位、可补数、可回放。

“at-least-once + 幂等写入 + 去重 + 可回放恢复”比空喊 exactly-once 更可靠。

在真实系统里，重复、重发、迟到、crash recovery 都不是罕见异常，而是运行语义。

01 Reliability

02 Batch

03 Incremental

04 Asset Flow

05 Recovery

能复现

同一批次能重放

能定位

run / state / source 可追踪

能补数

backfill 有边界

能解释

为什么重复、为什么不会漏

REPO REALITY

当前 repo 的增量主线

本周不要求本地搭完整 CDC 平台，但要把对象认清。

CONTRACT

`contracts/data/*.json`

字段与边界规则

MANIFEST

`data/seed_manifests/*.json`

load_mode / window / source

LOADER

`pipelines/ingestion/seed_loader.py`

manifest dry-run

TICKET

`pipelines/ingestion/ticket_ingest.py`

batch + cursor 思维

DOC

`pipelines/ingestion/doc_ingest.py`

document source ingest

TESTS

`tests/contract/test_json_schemas.py`

最小门禁

ADVANCED BOUNDARIES

进阶对象认知边界：知道意义，不急着一搭

logical decoding

变化从 WAL 来

replication slot / LSN

日志流位置与恢复边界

Debezium snapshot + stream

初始状态 + 后续变化

Kafka Connect EOS

有条件支持 exactly-once

streaming observability

持续状态与延迟观察

ENGINEERING HANDOFF

动手实践：把增量策略写进文档

这节课更多是设计与观察，不是搭重型 CDC。

```
mkdir -p docs/blueprints/week03
```

```
touch docs/blueprints/week03/incremental_ingest_strategy_v1.md  
touch docs/blueprints/week03/checkpoint_state_v1.md  
touch docs/blueprints/week03/late_arrival_decision_table.csv
```

建议先读

```
contracts/data/*.json  
data/seed_manifests/*.json  
pipelines/ingestion/seed_loader.py  
pipelines/ingestion/ticket_ingest.py
```

WRITE**策略说明**

cursor / watermark / checkpoint 怎么定义

WRITE**状态结构**

state 落在哪，谁更新

WRITE**决策表**

迟到、重复、乱序如何处理

COMPARISON

late arrival / duplicate / disorder 决策表

建议学生直接照这个骨架写 CSV。

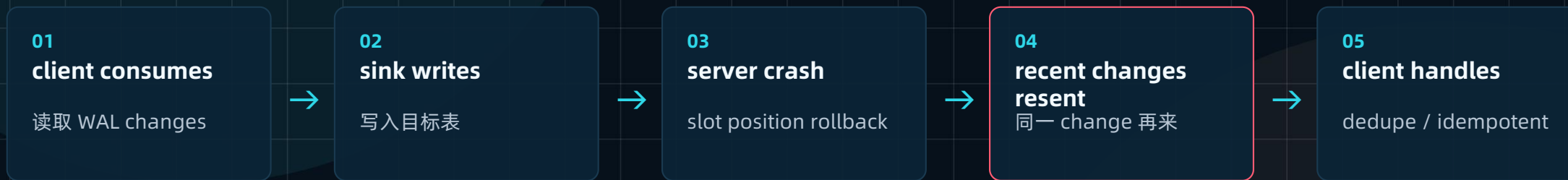
case	condition	action	evidence
late_in_window	event_time < watermark but within tolerance	accept + mark late	event_time / observed_at
late_out_of_window	beyond tolerance	quarantine / backfill review	window / reason_code
duplicate_input	same dedupe key	skip / merge	original_event_id
duplicate_write	same idempotency key	skip write	sink_key
semantic_drift	schema ok, meaning changed	quarantine + owner review	contract version

决策表就是 runbook 的前身。

CRASH RECOVERY

Crash 后为什么可能重发：slot 位置只在 checkpoint 持久化

用 PostgreSQL 官方语义解释“重复不是异常”。



屏幕上看起来像什么

- CDC 看起来“多发了一次”
- 下游多了一些重复记录
- 误以为 slot 不可靠

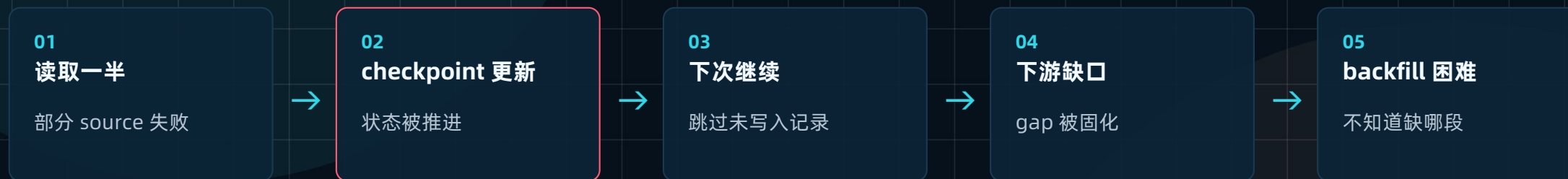
实际已经坏在哪

- 这是 documented behavior
- 客户端需要记录 LSN / dedupe
- 写入层必须幂等

STATE FAILURE CHAIN

状态写错，补数和回放都会失真

增量系统最怕 state 先于事实前进。



屏幕上看起来像什么

- 每次同步都成功
- 只是某些天少数据
- 监控不一定报错

实际已经坏在哪

- checkpoint 与写入事实不一致
- report 没有 source coverage
- 需要 restore/replay/backfill 判断

INDUSTRY SIGNALS

行业信号：生产级增量拼的是边界、状态和恢复

Airbyte

cursor + at-least-once

cursor 粒度与 updated_at 语义会带来重复或漏变更。

PostgreSQL

WAL / slot

logical decoding 从 WAL 抽取变化，crash 后可能重发最近 changes。

Debezium

EOS with boundaries

exactly-once 需要 Kafka Connect 条件，且仍需谨慎。

OpenLineage

run evidence

facets 可以把运行上下文对象化。

工具越复杂，越不能省掉状态和证据。

NOT A TOOL SHOW

不是 streaming 工具秀

不要让工具名掩盖工程边界。

不是

一上来堆 Kafka / Debezium 配置

不是

宣称 “已经 exactly-once”

不是

把 updated_at 当银弹

不是

只看消费 offset

而是

写清 cursor / watermark /
checkpoint

而是

接受 at-least-once +
idempotency

而是

为 replay/backfill 留证据

而是

能解释重复和缺口

高级不是工具多，而是边界诚实。

ANTI-PATTERNS

增量链路最常见的假动作

看起来工程化，实际上没有可恢复边界。

假动作	为什么危险	修正
只存 last_updated_at	不知道状态来自哪次 run	checkpoint 带 run_id / source / report
只看 offset	offset 不是业务事实	offset + dedupe + 业务 key
遇到重复就删	可能删除真实多版本	保留证据并定义 dedupe 规则
承诺 exactly-once	忽略系统边界和 crash 现实	讲清 at-least-once + 幂等
没有容忍窗口	迟到数据被静默丢弃	watermark + late policy

假动作的共同点：把恢复问题推给未来。

SELF-CHECK

自检：你能不能讲清这 6 个问题

面向小白和有经验学员都适用。

cursor 是什么

下一次从哪里继续读

watermark 是什么

已经确认处理到哪里

checkpoint 落在哪里

状态被谁持久化

dedupe key 怎么定

输入是否同一事件

idempotency key 怎么定

写入是否重复副作用

exactly-once 能否承诺

当前课程不承诺，边界要讲清

能讲清边界，就已经比很多“工具配置课”更接近生产。

RECAP & NEXT

增量与 CDC 的真正收口

从“少读一点”升级到“状态可解释”。

本课最重要的判断

- 增量不是天然可靠。
- cursor / watermark / checkpoint 不可混用。
- dedupe key / idempotency key 不可混用。
- CDC 不替代 snapshot, 而是 snapshot + change stream。

继续向前

- 有 slot 不等于不重不漏。
- 不要轻易承诺 exactly-once。
- 当前目标是 at-least-once + idempotent write + dedupe + replayable recovery。
- 下节课把 ingest 组织成资产流。

下一讲

Lesson 04 会把 manifest、ingest、state、metadata 组织到 Dagster asset / materialization / partition / backfill 视角。

ASSET FLOW · DAGSTER · PARTITIONS

4. 从任务流到资产流

任务跑完只是开始，资产可消费才是结果。

用 Dagster 的 asset / materialization / partition / backfill / asset check, 重新组织 Week03 ingest baseline。

Asset

Materialization

Partition

Backfill

Asset Check

Dagster UI

Week03 学习链

01 Reliability
为什么 ingest 可靠性决定下游

02 Batch
幂等写入、重跑与完整性校验

03 Incremental / CDC
cursor、watermark、
checkpoint

04 **Asset Flow**
用 Dagster 组织 ingest、分区与回
放

05 Recovery
Replay / Backfill / Runbook

WHAT THIS LESSON FIXES

这节课先解决什么问题

cron / shell script / 今天跑完就算成功，很快会失控。

WHY IT MATTERS

任务流的问题

- 只知道哪个脚本跑了。
- 不知道哪个资产已经产生。
- 不知道哪个分区缺失。
- 不知道哪个下游应该阻断。
- 出故障时只能围绕脚本名字补。

WHAT YOU CAN DO

资产流要回答

- manifest 产生了哪些资产。
- materialization 带了哪些 metadata。
- partition / batch window 如何定义。
- backfill 围绕哪个 asset + partition。
- asset check 如何守住健康状态。

本课目标：让 ingest 第一次有“资产图”。

讲义核心图：任务流为什么不够

同样的 manifest/source files，可以只进脚本，也可以进资产视角。



任务流回答“跑没跑”；资产流回答“什么数据资产可不可信”。

COMPARISON

为什么任务流不够

脚本层的成功，不等于数据资产层的成功。

任务流回答

哪个脚本跑了

哪个脚本失败

哪个任务慢

job exit code = 0

重跑某个脚本

数据工程真正关心

哪个资产已经产生

哪个资产分区缺失

哪个资产版本不可信

哪个下游资产应该阻断

补哪一个 asset + partition

生产里最重要的问题不是“任务有没有跑”，而是“资产是否可消费”。

Dagster 官方信号：asset 是一等公民

用官方定义支撑课程判断。

Asset

持久化对象

asset 是持久化存储中的对象，如 table、file、model。

Definition

代码描述应存在的资产

asset definition 描述资产应该存在以及如何生成/更新。

Materialization

产出一资产

materialize 是运行函数并把结果保存到持久化存储。

Assets vs ops

资产知道依赖

asset definition 知道 dependencies；ops 本身不天然知道。

Dagster 的价值不只是调度，而是让数据资产成为一等公民。

REPO REALITY

当前 repo 已经有资产化入口

这节课的正确姿势：读懂并扩展，不是大重构。

ASSETS**pipelines/ingestion/assets.py**

ingestion assets 定义

DEFS**pipelines/definitions.py**

统一注册 Definitions

ENTRY**seed_manifests**

manifest 资产入口

RAW**raw_doc_assets**

文档 raw 层落点

RAW**raw_ticket_events**

工单事件 raw 层落点

JOB**ingest_all_job**

触发最小 materialization

项目已经不是“只有脚本没有资产化入口”。

CORRECT POSTURE

这节课的正确姿势不是大重构

不要

重写 Dagster 全栈

不要

自己造新 orchestration

不要

立刻补全所有 sensors/schedules

不要

把脚本名当资产名

要

读懂资产图

要

识别已有资产

要

补 metadata / partition / policy

要

为 Week04 / Week06 接力

5 CONCEPTS

资产流的 5 个关键概念

概念	在 Week03 里最该抓住什么	典型误解
Asset	采集链路真正持续存在的结果对象	把函数当资产
Materialization	一次资产被成功产出的证据	把 job 成功当资产成功
Partition	未来增量、回放、补数的窗口/子集	等 Week06 再想
Backfill	对缺失/需重算的分区资产补跑	遇到问题全链路重跑
Asset Check	资产健康状态约束	只是另一个测试框架

先用这 5 个词重新描述 ingest baseline。

ASSET

Asset: 不是函数，而是持久化结果

seed_manifests / raw_ticket_events / raw_doc_assets 是资产，不只是步骤。

稳定 asset key

没有稳定 key, lineage / backfill / impact analysis 会混乱。

持久化结果

资产是表、文件、清单、模型等可被下游消费的结果。

依赖关系

资产视角天然关心谁依赖谁，任务视角不一定。

manifest 不是 asset; 它是这次 ingest 的声明。

MATERIALIZATION

Materialization: 不是任务跑了, 而是资产产出了一次

一次产出最好带着 metadata。

manifest_id

这次产出来自哪份声明

batch_id

对应哪个批次/窗口

run_id

哪次执行

source coverage

哪些 source 被覆盖

row count

写入/跳过/错误数量

checksum

来源或产出指纹

reject count

坏记录数量

reason code

失败或隔离原因

metadata 让 materialization 从“日志”变成“证据”。

PARTITION

Partition: 让 replay / backfill 有边界

分区不是只为大数据性能，也为恢复边界服务。

按天工单事件

ticket events daily partition

按 batch 文档清单

doc snapshot / manifest batch

按时间窗口 replay

只重放受影响窗口

按历史范围 backfill

补齐旧分区或重算

没有 partition 思维，backfill 很容易变成“重跑全链路”。

BACKFILL

Backfill: 不是“再跑一遍”

它是对缺失或需要重算的分区资产做有边界补跑。

补缺失分区

某日期/批次没有 materialize

重算旧分区

contract 或逻辑变化影响历史

控制影响范围

避免整链路重跑放大风险

围绕 asset

不是围绕脚本名字

围绕 partition

不是随便重跑全部数据

保留 evidence

补数动作要写进 report/runbook

backfill 的主语是 asset + partition。

ASSET CHECK

Asset Check: 资产健康约束，不是另一个口号

它验证资产的特定属性。

字段不能为空

关键字段 null ratio 低于阈值

记录数阈值

某窗口记录数不能异常低

schema 状态

字段 shape 与 contract 对齐

freshness

资产不能过期太久

quality

异常比例可解释

lineage

来源资产必须存在

PII boundary

敏感字段不得进入通用 serving

gate action

fail / warn / quarantine

后面很多 gate 会长成 asset check。

ASSET FLOW · REDRAW

讲义 asset flow: manifest 如何接住 ingest

seed manifest 进入资产入口，再分流到 raw_doc_assets / raw_ticket_events。



核心判断：manifest 描述想做什么，asset 表达实际产生了什么，job 只是触发方式。

THREE OBJECTS

manifest / asset / job 必须分开

混在一起，后面补数和恢复都会乱。

对象	回答的问题	典型错误
manifest	这次 ingest 想接什么、怎么接	把它当资产
asset	这次 ingest 实际产生了什么持久化结果	把函数名当资产名
job	用什么执行动作触发 materialization	把 job 成功当资产可消费
asset check	资产是否满足健康约束	只看脚本退出码

manifest 不是 asset, job 成功不等于下游可消费。

WHY NOW

为什么 partition / backfill 现在就要讲

不要等 Week06 才想恢复边界。

不先资产化

只知道脚本跑过，不知道资产是否可消费。

不先想 partition

replay/backfill 很难精准。

不带 metadata

lineage/runbook 会很虚。

不定 batch/window

补数容易多补、少补、重复补。

Week03 的资产边界，会直接决定 Week06 的恢复质量。

MATERIALIZATION METADATA

一次 materialization 应该挂哪些 metadata

让资产产出可解释、可对账、可恢复。

identity

run_id / batch_id / asset_key

manifest

manifest_id / contract_ref

source

source_id / source_fingerprint

coverage

source coverage / partition key

counts

row_count / reject_count / skipped

quality

check result / reason_code

state

checkpoint / watermark after

lineage

upstream / downstream assets

materialization metadata 是 run evidence 的资产化版本。

ENGINEERING HANDOFF

实践不会让你重写 Dagster，而是让你读懂资产图

先启动、跑最小 baseline，再看 Dagster UI。

```
docker compose --env-file infra/env/.env.local \
  -f infra/docker-compose.yml up -d --build

docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.ingestion.seed_loader \
  --manifest-dir data/seed_manifests

# 打开 Dagster UI
http://localhost:3000

# 记录
docs/blueprints/week03/asset_flow_plan_v1.md
docs/blueprints/week03/partition_backfill_strategy_v1.md
reports/week03/dagster_materialization_smoke_report.md
```

STEP 1

服务启动

确认基础服务可用

STEP 2

baseline

先跑最小 ingest

STEP 3

资产图

观察 ingestion assets

读懂资产图，是后续资产化的第一步。

DAGSTER UI OBSERVATION

看 Dagster UI 时不要只看绿色成功

要观察资产关系和 materialization evidence。

asset key

资产命名是否稳定

dependencies

谁依赖谁

materialization

本次产出是否有 metadata

partition

是否能按窗口管理

checks

是否可表达健康约束

run logs

能否追到 run_id

lineage

能否回到 manifest/source

downstream

失败是否应阻断下游

THREE JUDGMENTS

3 个工程判断

足够简单，也足够生产。

01

manifest 不是资产

它声明这次想做什么。资产是实际产生的持久化对象。

02

job 成功不等于下游可消费

必须看 materialization metadata 与 asset checks。

03

backfill 围绕 asset + partition

不要围绕脚本名字做补数。

这 3 句是 Lesson04 的核心。

DOWNSTREAM IMPACT

这节课为什么直接影响 Week04 和 Week06

资产边界会被后续持续消费。

Week04

Iceberg Bronze/Silver

需要稳定 asset 边界、partition 语义、run evidence，才能组织 lakehouse 层。

Week06

Replay / Backfill / Runbook

恢复动作要围绕 asset + partition，而不是围绕脚本名。

资产流不是 Dagster 专属知识点，而是后续课程的恢复底座。

INDUSTRY SIGNALS

行业信号：数据管道正在从任务日志走向资产运行证据

用官方资料收口到本课工程判断。

Dagster assets**asset / materialization**

资产是持久化对象，materialization 是一次资产产出。

Dagster partitions**partitions / backfills**

partition 支撑 incremental processing，backfill 面向缺失或需更新的分区资产。

Dagster checks**asset checks**

验证资产特定属性，适合表达数据健康约束。

OpenLineage**facets**

Run / Job / Dataset metadata 让运行证据对象化。

共同趋势：数据工程不再只看任务成功，而是看资产证据。

COMMON MISUNDERSTANDINGS

本课最容易误解的 5 件事

避免把 Dagster 学成另一个脚本调度器。

误解	正确理解
Dagster 就是调度工具	本课先用它建立资产视角。
asset 是函数	asset 是持久化结果，函数只是产生方式。
materialization = job 成功	materialization 是资产级产出证据。
backfill = 重新跑脚本	backfill 围绕 asset + partition。
现在不用想 partition	现在不想，Week06 会补不清。

任务流看执行，资产流看可消费结果。

ASSET FLOW PLAN

asset_flow_plan_v1.md 应该写什么

把本课判断沉淀进 repo。

已有资产

seed_manifests / raw_doc_assets /
raw_ticket_events

asset key

命名是否稳定，能否追踪

materialization metadata

manifest_id / batch_id / row_count

partition strategy

按天、按 batch、按 snapshot?

backfill boundary

哪些历史窗口能补

asset checks

字段、数量、freshness、quality

RECAP & NEXT

资产流完成了什么

从“任务是否成功”升级到“资产是否可消费”。

本课最重要的判断

- 任务成功不等于资产可消费。
- Asset 是持久化结果，不是函数。
- Materialization 是资产级产出证据。
- Partition 让 replay/backfill 有边界。

继续向前

- Asset Check 是资产健康约束。
- Manifest 描述想做什么，asset 表达实际产生什么。
- Backfill 必须围绕 asset + partition。
- 下节课讲故障恢复与 runbook。

下一讲

Lesson 05 会把 retry / rerun / replay / restore / backfill 拆开，并让 runbook 成为 Week03 的工程交付。

RECOVERY · REPLAY · RUNBOOK

5. 故障自愈与补数

系统跑通不是终点，出故障后能拉回正轨才是生产能力。

这节课把 retry / rerun / replay / restore / backfill 拆清楚，并把 Runbook 写成工程资产。

retry

rerun

replay

restore

backfill

runbook

Week03 学习链

- 01 Reliability
为什么 ingest 可靠性决定下游
- 02 Batch
幂等写入、重跑与完整性校验
- 03 Incremental / CDC
cursor、watermark、checkpoint
- 04 Asset Flow
用 Dagster 组织 ingest、分区与回放
- 05 Recovery
Replay / Backfill / Runbook

WHAT THIS LESSON FIXES

先解决什么问题

真实系统开始可运行之后，真正考验才开始。

WHY IT MATTERS

前四课已经推进到

- Week02 定义输入准入。
- Lesson01 建立 ingest baseline。
- Lesson02 做稳 batch 主链路。
- Lesson03 讲清增量状态。
- Lesson04 组织成资产流。

WHAT YOU CAN DO

这节课要回答

- 失败后 retry 还是 replay?
- backfill 是补历史空洞还是重算旧分区?
- 为什么不是全链路重跑?
- 谁判断、谁批准、谁执行、谁记录?
- runbook 为什么现在就要写?

核心判断：故障恢复不是重跑冲动，而是有边界、有依据、有记录的工程决策。

LECTURE DIAGRAM · REDRAW

讲义核心图：从异常信号到恢复报告

contract / manifest / state / run log 共同决定恢复动作。



恢复动作必须能解释：为什么是它，不是另外一个。

ACTION SPLIT

5 个动作先拆开：retry / rerun / replay / restore / backfill

它们不是同义词。

动作	真正解决什么	什么时候用	最容易被误用成
retry	执行级短暂失败	网络抖动、服务临时不可用	无脑重试
rerun	同一 job 再执行	上次执行中断	replay
replay	同一批输入重放	验证幂等、同批重走	新数据 ingest
restore	回到已知可用状态	下游/状态被污染	replay
backfill	补历史空洞或重算旧分区	历史缺口、逻辑变更	全链路重跑

恢复动作的主语不同，边界也不同。

RESTORE

restore 要单独看：它不是 replay，也不是 backfill

当状态或下游已被污染，先回到已知可用点。

Replay / Backfill

重新消费或重新计算某批/某段输入。

Restore

先回到已知可用快照、备份或安全状态。

为什么必要

对象存储、状态表、下游表若已污染，直接 replay 可能扩大问题。

遇到污染，先问当前状态能不能信。

MEMORY ANCHOR

记忆法：5 个动作对应 5 个恢复层级

帮助小白快速记住。

retry

执行级恢复

rerun

同 job 再执行

replay

输入级重放

restore

系统级回退

backfill

历史空洞修复

先判断层级，再决定命令。

WHY NOT FULL RERUN

为什么“出故障就全量重跑”不是成熟答案

全量重跑有时必要，但不能作为默认动作。

成本高

资源、时间、下游等待成本大。

风险大

可能覆盖或污染更多资产。

可解释性差

不知道到底修复了什么。

不利于 runbook

无法形成可复用恢复流程。

成熟团队默认做 **scoped recovery**，而不是全链路冲动。

REPO REALITY

当前 repo 已有恢复前提

恢复能力不是凭空出现，它依赖前面几课的锚点。

CONTRACT

`contracts/data/*.json`

准入与字段边界

MANIFEST

`data/seed_manifests/*.json`

输入批次声明

LOADER

`pipelines.ingestion.seed_loader`

dry-run / admission

ASSETS

`pipelines/ingestion/assets.py`

资产化入口

TESTS

`tests/contract/test_json_schemas.py`

契约门禁

REPORTS

`reports/week03/*.md`

恢复演练证据

没有这些前提，`replay/backfill` 只是脚本经验。

NOT FULLY AUTOMATED

当前还没有 fully automated, 这不是缺点

课程阶段要诚实区分“已有能力”和“未来能力”。

还没有

通用 replay service

还没有

自动 checkpoint/state manager

还没有

一键 backfill engine

还没有

recovery policy engine

已有

contract / manifest / dry-run

已有

asset entry / report 思维

已有

runbook 文档入口

已有

恢复决策边界

成熟不是假装全自动，而是把边界讲清、证据写实。

RECOVERY ANCHORS · REDRAW

恢复锚点图：contract + manifest + state + run log

恢复不是命令选择，而是锚点驱动的判断。



缺任何一个锚点，恢复就会变成猜。

ANCHORS EXPLAINED

恢复依赖哪些锚点

每个锚点都回答一个恢复问题。

Manifest

恢复哪类 source、哪个窗口、哪个来源清单。

State / Checkpoint

上次成功到哪里，replay 从哪里开始，backfill 是否覆盖缺口。

Run log / report

失败在哪层，后来采取什么动作，恢复是否成功。

runbook 不是凭经验写；它要引用这些锚点。

REPLAY VS BACKFILL

什么时候 replay，什么时候 backfill

用场景表替代口头感觉。

场景	推荐动作	为什么
网络抖动中断	retry / rerun	输入没变，执行层短暂失败
合法 manifest 想重走同一批验证幂等	replay	同批输入重放
某日期分区没入湖	backfill	补历史空洞
contract 变了需要重算历史	backfill	重算旧分区
replay 后仍少数据	查 state/run log，再决定 backfill	先定位缺口范围

恢复动作不是越大越安全，越精准越可控。

DECISION TREE · REDRAW

决策树：异常出现后先问当前状态能不能信

状态可信与否，决定先 restore 还是继续 scoped recovery。



决策树的价值：防止一上来全量重跑。

RECOVERY DRILL

Recovery drill: 本课准备 3 个文件

把恢复能力写成可提交工件。

runbooks/ingestion_runbook_v1.md

恢复流程与升级路径

reports/week03/recovery_drill_report.md

演练过程与结果证据

docs/blueprints/week03/replay_backfill_strategy_v1.md

replay/backfill 决策依据

这三个文件让“恢复”从经验变成可复盘资产。

ENGINEERING HANDOFF

Drill 第一步：确认 baseline 可跑

这不是重复实验，而是确认干净起点。

```
docker compose --env-file infra/env/.env.local \
  -f infra/docker-compose.yml up -d --build
```

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  pytest tests/contract/ -v
```

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.ingestion.seed_loader \
  --manifest-dir data/seed_manifests
```

BASELINE**契约通过**

准入标准可用

BASELINE**manifest 可读**

批次声明存在

BASELINE**dry-run 有证据**

run evidence 起点

先确认干净起点，再定义故障场景。

DEFINE SCENARIO

Drill 第二步：人为定义恢复场景

不要直接动手跑，先定义故障。

manifest 跳过一个 source

适合 scoped replay

历史批次需要补回

适合 backfill 边界定义

contract 结构合法但历史逻辑需重算
适合逻辑变更后的 recovery 说明

恢复演练要先写场景，不要先跑命令。

DECISION FIRST

先写恢复决策，不要直接动手跑

replay_backfill_strategy_v1.md 至少写 6 项。

场景描述

发生了什么

当前症状

看到了哪些信号

目标动作

retry / replay / backfill / restore

为什么不是另外几个

排除其他动作的理由

输入边界

manifest / source / window / partition

验收标准

恢复后如何证明成功

写决策，是为了防止恢复动作变成个人经验。

RUNBOOK TEMPLATE

Runbook 模板：不是最后才写

它逼你暴露缺失能力。

模块	要写什么	讲师提醒
触发条件	什么信号出现就进入 runbook	不要只写“失败时”
前置检查	contract / manifest / state / report	先查锚点
执行命令	统一 Docker-first 命令	命令要可复制
风险提醒	会影响哪些资产/分区	防止扩大影响
验证方式	counts / reconcile / checks	恢复要能证明
升级路径	何时人工 review / owner 介入	不要硬扛

好的 runbook 是工程能力，不是运维附录。

RECOVERY REPORT

recovery_drill_report 应该记录什么

恢复报告是 run evidence 的后续。

采用什么动作

retry / replay / restore / backfill

为什么这么选

引用状态、manifest、run log

执行了哪些命令

命令与时间点

预期 vs 实际

恢复后指标是否闭合

验证结果

reconcile / row counts / checks

尚未自动化能力

下一步工程债

没有恢复报告，runbook 就无法迭代。

WHY EARLY RUNBOOK

Runbook 为什么不是最后才写

逼你讲清术语

retry/rerun/replay/backfill 不再混用。

暴露缺失能力

没有 state、report、command，就写不出来。

缩短恢复时间

把个人经验变成团队路径。

连接治理闭环

接 Week12 tracing 与 Week14 governance。

runbook 不是文档负担，而是生产能力的压力测试。

恢复动作还需要角色边界：谁判断、谁批准、谁执行、谁记录

生产里恢复失败，常常不是技术不会，而是责任不清。

判断者

根据 report/state 判断动作

批准者

确认影响范围与风险

执行者

按 runbook 执行命令

记录者

写 recovery report

Owner

解释 source/contract 语义

Reviewer

复核补数结果

Escalation

超出权限或影响范围升级

Auditor

后续追踪与治理

INDUSTRY SIGNALS

行业信号：恢复动作正在从脚本经验走向有边界的运行资产

用 Dagster 与 OpenLineage 收口本课。

Dagster

partitions / backfills

backfill 围绕分区资产，而不是随意重跑脚本。

Dagster

asset checks

恢复后可以用资产健康约束验证结果。

OpenLineage

facets

run evidence 可记录恢复上下文与影响范围。

Course stance

runbook as asset

恢复流程本身也是可复用、可审计的工程资产。

恢复越成熟，越像“有边界的资产操作”，而不是“脚本补锅”。

RECAP & NEXT

Week03 收官：把链路拉回正轨的 8 个判断

这周完成的是 ingest baseline，不是空泛 ETL。

本课最重要的判断

- retry / rerun / replay / restore / backfill 不是同义词。
- recovery 是工程决策，不是重跑冲动。
- 没有 contract / manifest / state / run log，就无法做可解释恢复。
- replay 是重放同一批输入。

继续向前

- backfill 是补历史空洞或重算历史分区。
- Week03 先把恢复边界讲清，不是假装 fully automated。
- Dagster 的 partition/backfill 思维现在就要进入心智。
- 好的 Runbook 是 Week03 最重要交付物之一。

Week03 完整交付

ingestion_baseline_v1.md、checkpoint_state_v1.md、
ingestion_runbook_v1.md、recovery_drill_report.md、
state/report/replay/backfill 观察记录。

CLOSING BRIDGE

下一步：把可靠 ingest baseline 交给 lakehouse 与 orchestration

Week03 做得越扎实，后面的索引、检索、评测和治理越稳。

Week04**Lakehouse**

Bronze/Silver/Iceberg 需要稳定 ingest 结果。

Week06**Orchestration**

replay/backfill/runbook 继续自动化。

Week08**RAG evidence**

run evidence 支撑 citation 与 evidence serving。

Week11+**Eval / Governance**

trace、state、release 进入评测与治理闭环。

Week03 的核心不是“搬数据”，而是让数据进入系统后可解释、可恢复、可治理。